Календарь и стартовая страница в Docker

Инструкция администратора · сервер docker.domain.ru (10.x.x.x, РЕД ОС 8) · редакция от 01.11.2026

1. Как устроено
2. Файлы на сервере
3. Вход на сервер
4. Повседневные команды
5. Первый запуск с нуля
6. Настройки календаря (.env)
7. Обновление календаря
8. Стартовая страница и nginx
9. Служебные команды календаря
10. Резервное копирование и восстановление
11. Возврат к старой схеме
12. Диагностика
13. Безопасность

> **Что изменилось в редакции от 01.11.2026.** Сервер получил адрес **10.10.189.14** (был 10.10.161.21), записи DNS обновлены. **Доступ к Docker Hub закрыт**: обычная сборка образа календаря (`docker compose up -d --build`) и `docker compose pull` не работают — обновление календаря выполняется сборкой поверх текущего образа (раздел 7).

1. Как устроено

На сервере работают два контейнера, оба описаны в одном файле `/opt/proxy/docker-compose.yml` :

Контейнер	Образ	Назначение
`proxy-proxy-1` (служба `proxy`)	`nginx:stable`	Единственный, кто слушает порт 80. Разводит запросы по имени сайта: `start.zs.int` — отдаёт файлы стартовой страницы, `calendar.zs.int` — передаёт запрос календарю. Запросы по IP-адресу и с неизвестным именем закрывает без ответа.
`proxy-calendar-1` (служба `calendar`)	`calendar` (собирается из `/opt/calendar`)	Приложение «Календарь» (Django + gunicorn, порт 8000 только внутри сети Docker). При старте обновляет структуру базы, каждые 10 минут синхронизирует внешние календари и досылает приглашения.

```
браузер ──► :80 proxy (nginx) ─┬─ start.zs.int    ──► /opt/start/start.zs.int (файлы)
                               └─ calendar.zs.int ──► calendar:8000 ──► /opt/calendar/data
                                                                        (db.sqlite3 и media/)
```

Оба контейнера запускаются с политикой `restart: always` : после перезагрузки сервера или сбоя они поднимаются сами. Имена `start.zs.int` и `calendar.zs.int` в DNS указывают на **10.10.189.14**.

2. Файлы на сервере

Путь	Что это
`/opt/proxy/docker-compose.yml`	Описание обоих контейнеров. Все команды `docker compose` выполняются из `/opt/proxy`.
`/opt/proxy/conf.d/00-default.conf`	Сайт «по умолчанию»: закрывает запросы по IP и с чужим именем.
`/opt/proxy/conf.d/start.zs.int.conf`	Настройки стартовой страницы (`users.html` / `admins.html` для сети 10.10.160.0/24).
`/opt/proxy/conf.d/calendar.zs.int.conf`	Передача запросов календарю; лимит загрузки файлов 420 МБ.
`/opt/proxy/conf.d/hr.zs.int.conf`	Сайт `hr.zs.int` (`hr.zsdomain.ru`), добавлен после редакции от 14.10.2026; в этой инструкции не описан.
`/opt/start/start.zs.int/`	Файлы стартовой страницы. Правка применяется сразу, без перезапуска.
`/opt/calendar/`	Код календаря и `Dockerfile`; владелец — `zsadmin`.
`/opt/calendar/.env`	Настройки и секреты календаря (права 600). **Не терять и не публиковать.**
`/opt/calendar/data/db.sqlite3`	База: пользователи, события, подключения, настройки почты и каталогов.
`/opt/calendar/data/media/`	Файлы, прикреплённые к событиям.
`/opt/backup/`	Резервные копии (доступ только root). `2026-10-13_1753` — копия до перехода на эту схему.
`/opt/backup/2026-10-14_start-cleanup`	Убранные 14.10.2026 из `/opt/start` файлы старой схемы (Dockerfile, `update_container.sh`, копия конфигурации nginx) и 39 картинок, на которые не ссылаются страницы.
`/home/zsadmin/backups/`	Резервные копии, сделанные без root (раздел 10.1). `2026-11-01_0932` — копия перед обновлением 01.11.2026.
`/opt/calendar/docker-compose.yml`	Файл из поставки календаря для запуска «отдельно». На этом сервере **не используется**.

Данные календаря принадлежат пользователю с uid 1000 (`zsadmin`) — под этим uid работает приложение в контейнере. На сервере включён SELinux, поэтому каталоги подключены с метками `:Z` / `:z`; не убирайте их из `docker-compose.yml`.

Образы на сервере: `calendar` — рабочий образ календаря, `calendar:prev` — предыдущий, для отката (раздел 7), `nginx:stable`.

3. Вход на сервер

```
ssh zsadmin@10.10.189.14
sudo -i                 # команды docker выполняются от root (zsadmin не входит в группу docker)
cd /opt/proxy
```

Все команды ниже, если не сказано иное, выполняются от root в каталоге `/opt/proxy`.

4. Повседневные команды

Команда	Действие
`docker compose ps`	Состояние контейнеров. Норма: оба `Up`, у календаря `(healthy)`.
`docker compose logs -f --tail 100`	Журналы обоих контейнеров в реальном времени (Ctrl+C — выход).
`docker compose logs --since 1h calendar`	Журнал календаря за последний час.
`docker compose logs proxy | grep " 50[0-9] "`	Ошибки 5xx в журнале nginx.
`docker compose restart calendar`	Перезапуск календаря (настройки из `.env` так **не** перечитываются — см. раздел 6).
`docker compose restart proxy`	Перезапуск nginx (оба сайта недоступны несколько секунд).
`docker compose up -d`	Поднять всё, что остановлено, и пересоздать контейнеры, у которых изменились настройки.
`docker compose stop` / `docker compose start`	Остановить / запустить оба контейнера (данные сохраняются).
`docker compose down`	Остановить и удалить контейнеры (данные в `/opt/calendar/data` не затрагиваются). Вернуть — `docker compose up -d`.
`docker stats --no-stream`	Загрузка процессора и памяти контейнерами.
`df -h /opt`, `du -sh /opt/calendar/data`	Свободное место и объём данных календаря.

Журналы контейнеров пишутся в системный журнал (journald), поэтому их видно и так: `journalctl CONTAINER_NAME=proxy-calendar-1 --since today`.

Быстрая проверка обоих сайтов прямо с сервера:

```
curl -sI -H 'Host: start.zs.int'    http://127.0.0.1/ | head -1   # HTTP/1.1 200 OK
curl -sI -H 'Host: calendar.zs.int' http://127.0.0.1/ | head -1   # HTTP/1.1 302 Found (на страницу входа)
```

5. Первый запуск с нуля

Раздел нужен при переносе на новый сервер или после полной потери системы. На текущем сервере всё уже настроено.

> **Без доступа к Docker Hub** первый запуск с нуля невозможен: базовые образы `python:3.12-slim` и `nginx:stable` и пакеты Python скачиваются из интернета. Нужен временный доступ к Docker Hub и PyPI (или зеркало), либо перенос готовых образов с работающего сервера: `docker save calendar nginx:stable | gzip > images.tar.gz`, на новом сервере — `docker load < images.tar.gz`.

5.1. Docker

```
dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin   # или пакеты из репозитория РЕД ОС
systemctl enable --now docker
docker compose version
```

5.2. Код календаря

Код копируется с рабочей машины, где лежит репозиторий (git на сервере не установлен). Выполняется на рабочей машине из каталога `main` репозитория:

```
rsync -av --delete \
  --exclude .env --exclude 'data/' --exclude db.sqlite3 --exclude 'media/' \
  --exclude local_settings.py --exclude '__pycache__/' --exclude 'staticfiles/' --exclude '.venv/' \
  ./ zsadmin@10.10.189.14:/opt/calendar/
```

5.3. Данные и настройки

```
mkdir -p /opt/calendar/data/media
chown -R 1000:1000 /opt/calendar/data
cp /opt/calendar/.env.example /opt/calendar/.env      # заполнить, см. раздел 6
chown 1000:1000 /opt/calendar/.env && chmod 600 /opt/calendar/.env
```

При переносе со старого сервера вместо пустого каталога положите в `/opt/calendar/data` файл `db.sqlite3` и каталог `media/`, а `.env` возьмите старый — с теми же ключами (иначе сохранённые пароли подключений не расшифруются).

5.4. nginx

Создайте `/opt/proxy/docker-compose.yml` и три файла в `/opt/proxy/conf.d/` — их текст приведён в приложении в конце документа.

5.5. Запуск

```
cd /opt/proxy
docker compose up -d --build           # первая сборка образа календаря — 1–3 минуты (нужен Docker Hub и PyPI)
docker compose ps                       # через ~30 с у календаря должно быть (healthy)
docker compose exec calendar python manage.py createsuperuser   # только для новой пустой базы
```

Локальный суперпользователь — резервный вход, если недоступен сервер домена. Обычные пользователи входят доменными учётными записями.

6. Настройки календаря (.env)

Файл `/opt/calendar/.env`, формат `ИМЯ=значение`. Основные параметры:

Параметр	Назначение
`DJANGO_SECRET_KEY`	Секретный ключ приложения. Смена ключа завершит все сеансы пользователей.
`INTEGRATIONS_ENCRYPTION_KEY`	Ключ шифрования сохранённых паролей (внешние календари, почта, каталоги). **Не менять**: без него пароли в базе не расшифровать.
`DJANGO_DEBUG`	Должно быть `0`. Дополнительно принудительно задано в `docker-compose.yml`.
`DJANGO_ALLOWED_HOSTS`	Имена, по которым открывают календарь: сейчас `10.10.161.21,calendar.zs.int`. Старый IP ни на что не влияет — запросы по IP nginx закрывает (раздел 1); работает имя `calendar.zs.int`. Новое имя сайта нужно добавить сюда и в `calendar.zs.int.conf` (`server_name`).
`DJANGO_CSRF_TRUSTED_ORIGINS`	Нужен только при работе по HTTPS: `https://calendar.zs.int`.
`AD_AUTH_*`	Вход через домен: домен, серверы, шифрование, группы доступа и администраторов.
`CALENDAR_MANAGERS`	Логины пользователей с правами управляющего календарём.
`EMAIL_*`, `DEFAULT_FROM_EMAIL`, `INTEGRATIONS_MAIL_FROM_USER`	Почтовый сервер для приглашений (может быть переопределён в админке).
`INTEGRATIONS_EXCHANGE_URL`, `INTEGRATIONS_UID_DOMAIN`, `GOOGLE_OAUTH_*`	Внешние календари.

Подробное описание каждого параметра — в `ADMIN_GUIDE.md` (раздел 3) в составе календаря.

> После изменения `.env` контейнер нужно **пересоздать**, а не просто перезапустить — `restart` оставляет старые значения:
>
> `bash cd /opt/proxy && docker compose up -d --force-recreate calendar`

Интервал синхронизации внешних календарей задаётся в `docker-compose.yml`: `SYNC_INTERVAL_MINUTES` (по умолчанию 10, `0` — отключить). После правки — `docker compose up -d`

`calendar`.

7. Обновление календаря

Доступ к Docker Hub закрыт, поэтому обычная пересборка (`docker compose up -d --build calendar`) не проходит: она скачивает базовый образ `python:3.12-slim`. Новый код ставится поверх текущего рабочего образа — заменяются только код и статика, Python и пакеты остаются прежними.

1. Сделайте резервную копию базы (раздел 10.1).
2. На рабочей машине скопируйте новый код командой `rsync` из раздела 5.2. Исключения в команде защищают `.env` и данные от удаления — не убирайте их.
3. На сервере соберите образ поверх текущего и пересоздайте контейнер:

```
bash cd /opt/proxy docker tag calendar calendar:prev # прежний образ — для отката cat > /tmp/Dockerfile.offline <<'EOF' FROM calendar:prev USER root COPY . /app/ RUN cd /app && python manage.py collectstatic --noinput && chmod +x docker-entrypoint.sh USER calendar EOF DOCKER_BUILDKIT=0 docker build -f /tmp/Dockerfile.offline -t calendar /opt/calendar docker compose up -d --no-build calendar docker compose logs --tail 50 calendar # миграции базы и запуск gunicorn docker compose ps # (healthy)
```

`DOCKER_BUILDKIT=0` (старый сборщик) нужен, чтобы сборка не обращалась к реестру образов; предупреждение `legacy builder is deprecated` можно не читать.

Откат на предыдущую версию: `docker tag calendar:prev calendar && docker compose up -d --no-build calendar` (код в `/opt/calendar` при этом остаётся новым — верните его из копии `calendar-code.tar.gz`, раздел 10.1, или повторным `rsync` старой версии).

Ограничение: если в новой версии изменился набор пакетов (`requirements.txt`, `requirements-docker.txt`), такой способ их не установит — нужен временный доступ к PyPI или заранее скачанные пакеты. Удалённые из кода файлы остаются в образе — это безвредно, при следующей полной сборке они исчезнут.

Изменения структуры базы применяются автоматически при старте контейнера. Перерыв в работе календаря — 10–20 секунд; стартовую страницу обновление не затрагивает. Пользователям чистить кэш браузера не нужно.

Обновление nginx (`docker compose pull proxy && docker compose up -d proxy`) требует доступа к Docker Hub и сейчас невозможно.

Очистка старых образов после обновлений (рабочие образы не удаляются): `docker image prune -f`. Образ `calendar:prev` она не удаляет — у него есть имя.

8. Стартовая страница и nginx

8.1. Содержимое стартовой страницы

Файлы в `/opt/start/start.zs.int/` (`users.html`, `admins.html`, картинки и т. п.) редактируются на месте; изменения видны сразу после обновления страницы в браузере. Пересобирать контейнеры не нужно. Картинку, которую понадобится вернуть, ищите в `/opt/backup/2026-10-14_start-cleanup/img`.

Новые файлы, скопированные в этот каталог, получают нужную метку SELinux автоматически. Если новый файл отдаётся с ошибкой 403, восстановите метку: `chcon -R -t container_file_t /opt/start/start.zs.int`.

8.2. Изменение настроек nginx

После правки любого файла в `/opt/proxy/conf.d/` проверьте и примените настройки без перерыва в работе:

```
docker compose exec proxy nginx -t              # syntax is ok / test is successful
docker compose exec proxy nginx -s reload
```

Если `nginx -t` сообщил об ошибке, **не перезапускайте контейнер** — исправьте файл: при ошибке в настройках nginx не запустится и оба сайта станут недоступны.

8.3. Новый сайт на этом же сервере

1. Добавьте в DNS запись `имя.zs.int → 10.10.189.14`.
2. Создайте `/opt/proxy/conf.d/имя.zs.int.conf` с блоком `server { listen 80; server_name имя.zs.int; … }` — по образцу `start.zs.int.conf` (статические файлы; каталог подключите в `docker-compose.yml` строкой в `volumes` службы `proxy` с меткой `:ro,z`) или `calendar.zs.int.conf` (приложение в отдельном контейнере, добавленном в тот же `docker-compose.yml`).
3. Если менялся `docker-compose.yml` — `docker compose up -d`, иначе — `nginx -t` и `nginx -s reload` (раздел 8.2).

9. Служебные команды календаря

Команды выполняются внутри работающего контейнера: `docker compose exec calendar python manage.py КОМАНДА`.

Команда	Что делает
`check`	Проверка настроек приложения.
`createsuperuser`	Создать локального администратора (резервный вход без домена).
`changepassword ЛОГИН`	Сменить пароль локальной учётной записи.
`ad_auth_check --diagnose`	DNS и доступность серверов домена (LDAP-порты).
`ad_auth_check ЛОГИН`	Проверка входа конкретного пользователя (запросит пароль).
`directory_check [--search ТЕКСТ] [--diagnose]`	Проверка адресной книги.
`exchange_check ID_СОБЫТИЯ [--fix]`	Как событие выглядит в Exchange; `--fix` — перезаписать и дослать файлы.
`sync_calendars [--user ЛОГИН]`	Синхронизировать внешние календари сейчас, не дожидаясь планового запуска.

Для команд, которые спрашивают пароль, нужен интерактивный терминал — запускайте их из сеанса `ssh`, а не из скрипта. Административный веб-интерфейс: http://calendar.zs.int/admin/.

10. Резервное копирование и восстановление

Сохранять нужно три вещи — **вместе**: `/opt/calendar/data/db.sqlite3`, `/opt/calendar/data/media/` и `/opt/calendar/.env`. База без ключей из `.env` бесполезна для подключений: сохранённые пароли не расшифровать. Конфигурацию `/opt/proxy` и стартовую страницу `/opt/start` полезно копировать туда же.

10.1. Копия без остановки календаря

От root:

```
d=/opt/backup/$(date +%F_%H%M); mkdir -p "$d" && chmod 700 "$d"
cd /opt/proxy
docker compose exec -T calendar python -c \
  "import sqlite3;
sqlite3.connect('/data/db.sqlite3').backup(sqlite3.connect('/data/backup.sqlite3'))"
mv /opt/calendar/data/backup.sqlite3 "$d/db.sqlite3"
tar -C /opt/calendar/data -czf "$d/media.tar.gz" media
cp -p /opt/calendar/.env "$d/.env"
tar -czf "$d/config.tar.gz" /opt/proxy /opt/start
ls -la "$d"
```

Без root (от `zsadmin`, владельца данных календаря) — копия базы, файлов, `.env` и кода в домашний каталог:

```
d=~/backups/$(date +%F_%H%M); mkdir -p "$d"; chmod 700 ~/backups "$d"
python3 -c "import sqlite3,sys;
sqlite3.connect('/opt/calendar/data/db.sqlite3').backup(sqlite3.connect(sys.argv[1]))"
"$d/db.sqlite3"
tar -C /opt/calendar/data -czf "$d/media.tar.gz" media
cp -p /opt/calendar/.env "$d/.env"
tar -C /opt -czf "$d/calendar-code.tar.gz" --exclude=calendar/data --exclude=calendar/.env
calendar
ls -la "$d"
```

Копию базы нельзя делать простым `cp` работающего файла — можно получить испорченную копию; используйте команды выше или остановите календарь (`docker compose stop calendar`). Копии в `/opt/backup` и `~/backups` лежат на том же диске — регулярно переносите их на другой сервер или в хранилище. Старые копии удаляйте вручную: `ls /opt/backup`, затем `rm -r /opt/backup/ДАТА`.

Автоматически каждую ночь: сохраните команды первого блока в `/usr/local/sbin/calendar-backup.sh` (`chmod 700`), последняя строка (`ls`) там не нужна. Затем добавьте задание (`crontab -e` от root):

```
30 2 * * * /usr/local/sbin/calendar-backup.sh >/var/log/calendar-backup.log 2>&1
0 3 * * * find /opt/backup -mindepth 1 -maxdepth 1 -type d -mtime +30 -exec rm -r {} +
```

10.2. Восстановление

```
cd /opt/proxy
docker compose stop calendar
cp /opt/backup/ДАТА/db.sqlite3 /opt/calendar/data/db.sqlite3
rm -rf /opt/calendar/data/media && tar -C /opt/calendar/data -xzf /opt/backup/
ДАТА/media.tar.gz
cp -p /opt/backup/ДАТА/.env /opt/calendar/.env          # если менялись ключи
chown -R 1000:1000 /opt/calendar/data /opt/calendar/.env
docker compose up -d --force-recreate calendar
```

Для копии из `~zsadmin/backups` подставьте в команды путь `/home/zsadmin/backups/ДАТА`.

В копии `2026-10-13_1753` (до перехода на Docker-схему) база называется `db.sqlite3`, файлы лежат в каталоге `media`, а `.env` относится к старой установке: для восстановления из неё используйте текущий `.env`.

11. Возврат к старой схеме

Если новая схема неработоспособна и быстро исправить её не удаётся, стартовую страницу можно вернуть на старый контейнер (образ `start_page` сохранён). Календарь при этом будет недоступен.

```
cd /opt/proxy && docker compose down
docker run -d --name start.zs.int -p 80:80 --restart=always start_page
```

Вернуться к новой схеме: `docker rm -f start.zs.int && cd /opt/proxy && docker compose up -d`.

12. Диагностика

Признак	Причина и действие
Сайт не открывается по имени ни у кого	`docker compose ps`; проверка с сервера командами `curl` из раздела 4. Если с сервера всё отвечает — проверьте DNS: `nslookup start.zs.int 10.10.161.111` должен вернуть 10.10.189.14.
Не открывается только у одного пользователя	Проблема на рабочем месте: имя не разрешается (`nslookup start.zs.int`). Частая причина — включённый VPN, который подменяет DNS. Отключите VPN или добавьте в `hosts` строку `10.10.189.14 start.zs.int calendar.zs.int`. Если в `hosts` осталась строка со старым адресом 10.10.161.21 — исправьте её.
Соединение сбрасывается, «пустой ответ» при открытии по IP	Так и задумано: сайты отвечают только по именам `start.zs.int` и `calendar.zs.int`.

Признак	Причина и действие
Порт 80 занят, `proxy` не стартует	Запущен старый контейнер: `docker ps -a`, удалите `start.zs.int` (`docker rm -f start.zs.int`), затем `docker compose up -d`.
`calendar.zs.int`: «502 Bad Gateway»	Календарь не запущен или перезапускается: `docker compose ps`, `docker compose logs --tail 100 calendar`.
`calendar.zs.int`: «Bad Request (400)»	Имя сайта не указано в `DJANGO_ALLOWED_HOSTS` (раздел 6).
«413 Request Entity Too Large» при загрузке файлов	Превышен `client_max_body_size` в `calendar.zs.int.conf`.
Сборка образа: `failed to resolve source metadata for docker.io/library/python…`, `i/o timeout`	Нет доступа к Docker Hub. Обновляйте календарь сборкой поверх текущего образа (раздел 7). Работающий контейнер при неудачной сборке не страдает.
Календарь постоянно перезапускается	`docker compose logs calendar`. Типичные причины: ошибка в `.env`; нет прав на `/opt/calendar/data` (`chown -R 1000:1000 /opt/calendar/data`); закончилось место (`df -h`). После обновления — откат на `calendar:prev` (раздел 7).
«Permission denied» при доступе к файлам в журналах	SELinux: проверьте метки `:Z` / `:z` в `docker-compose.yml`; отказы видны в `ausearch -m avc -ts recent`.
Состояние `(unhealthy)`	Приложение не отвечает внутри контейнера: `docker inspect --format '{{json .State.Health}}' proxy-calendar-1` и журнал календаря.
Никто не может войти в календарь	`docker compose exec calendar python manage.py ad_auth_check --diagnose`; войдите локальным суперпользователем.
Приглашения не уходят, внешние календари не синхронизируются	Журнал календаря (`sync_calendars` пишет итог каждые 10 минут); тестовое письмо в админке; `sync_calendars` вручную (раздел 9).
После изменения `.env` ничего не изменилось	Контейнер перезапущен, а не пересоздан: `docker compose up -d --force-recreate calendar`.

13. Безопасность

- `/opt/calendar/.env` — права 600, содержит ключи и пароли. Не отправляйте его по почте и не кладите в репозиторий.
- Каталоги `/opt/backup` (только root) и `/home/zsadmin/backups` (только `zsadmin`) содержат базу и ключи — храните их внешние копии так же надёжно.
- Порт календаря 8000 наружу не открыт: календарь доступен только через nginx.
- Сайты работают по HTTP; пароли пользователей домена передаются по сети в открытом виде. При переходе на HTTPS понадобятся сертификат, блок `listen 443 ssl` в nginx, проброс порта 443 в `docker-compose.yml`, а в `.env` — `DJANGO_BEHIND_PROXY=1` и `DJANGO_CSRF_TRUSTED_ORIGINS=https://calendar.zs.int`.

- Периодически обновляйте систему сервера. Образ nginx без доступа к Docker Hub не обновить (раздел 7).

Приложение. Файлы конфигурации

`/opt/proxy/docker-compose.yml`

```
services:
  proxy:
    image: nginx:stable
    restart: always
    ports:
      - "80:80"
    volumes:
      - ./conf.d:/etc/nginx/conf.d:ro,Z
      - /opt/start/start.zs.int:/usr/share/nginx/start.zs.int:ro,z
    depends_on:
      - calendar

  calendar:
    build: /opt/calendar
    image: calendar
    restart: always
    env_file: /opt/calendar/.env
    environment:
      DJANGO_DEBUG: "0"
      SYNC_INTERVAL_MINUTES: "10"
    volumes:
      - /opt/calendar/data:/data:Z
```

`/opt/proxy/conf.d/00-default.conf`

```
server {
    listen 80 default_server;
    server_name _;
    return 444;
}
```

`/opt/proxy/conf.d/start.zs.int.conf`

```
geo $index_file {
    default "users.html";
    10.10.160.0/24 "admins.html";
}

server {
    listen 80;
    server_name start.zs.int;

    location / {
        try_files $uri $uri/ /$index_file;
        root /usr/share/nginx/start.zs.int;
        index $index_file;
    }

    location /admins {
        allow 10.10.160.0/24;
        deny all;
        root /usr/share/nginx/start.zs.int;
        index $index_file;
    }
}
```

`/opt/proxy/conf.d/calendar.zs.int.conf`

```
server {
    listen 80;
    server_name calendar.zs.int;
    client_max_body_size 420m;

    resolver 127.0.0.11 valid=10s;
    set $calendar http://calendar:8000;

    location / {
        proxy_pass $calendar;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_read_timeout 120s;
    }
}
```